

P3395R4

Fix encoding issues and add a formatter for `std::error_code`

Published Proposal, 2025-04-27

Author:

[Victor Zverovich](#)

Audience:

LEWG

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Table of Contents

- 1 Introduction**
- 2 Changes since R3**
- 3 Changes since R2**
- 4 Changes since R1**
- 5 Changes since R0**
- 6 Polls**
- 7 Motivation**
- 8 Proposal**
- 9 Previous work**
- 10 Wording**

11 Implementation

References

Informative References

§ 1. Introduction

This paper proposes making `std::error_code` formattable using the formatting facility introduced in C++20 (`std::format`) and fixes encoding issues in the underlying API ([\[LWG4156\]](#)).

§ 2. Changes since R3

- Added LEWG poll results for R3.
- Fixed a typo in the wording.
- Fixed wording for the debug output.
- Clarify why the format specifier for value is not provided.

§ 3. Changes since R2

- Added a reference to [\[P2930\]](#) and how it differs from the current proposal.

§ 4. Changes since R1

- Added a debug format to avoid ambiguity when formatting error codes in maps.
- Added SG16 poll results.

§ 5. Changes since R0

- Changed the title from "Formatting of `std::error_code`" to "Fix encoding issues and add a formatter for `std::error_code`" to reflect the fact that the paper also fixes [\[LWG4156\]](#).
- Specified that `error_category::name()` returns a string the ordinary literal encoding per SG16 feedback.

- Made transcoding in `error_category::message()` implementation-defined if the literal encoding is not UTF-8 per SG16 feedback and for consistency with other similar cases in the standard.

§ 6. Polls

LEWG poll results for R3:

POLL: P3395 should explore format specifier support to define which information (error number/category/message) to format.

SF	F	N	A	SA
1	9	4	2	0

Outcome: Consensus in favour

SG16 poll results for R0:

POLL: Forward P3395R0 to LEWG amended to specify an encoding for `std::error_category::name()` and for transcoding to be to UTF-8 if that matches the ordinary literal encoding and to an implementation-defined encoding otherwise.

SF	F	N	A	SA
1	6	0	0	0

Outcome: Strong consensus

§ 7. Motivation

`error_code` has a rudimentary `ostream` inserter. For example:

```
std::error_code ec;
auto size = std::filesystem::file_size("nonexistent", ec);
std::cout << ec;
```

This works and prints `generic:2`.

However, the following code doesn't compile:

```
std::print("{}\n", ec);
```

Unfortunately, the existing inserter has several issues, such as I/O manipulators applying only to the category name rather than the entire error code, resulting in confusing output:

```
std::cout << std::left << std::setw(12) << ec;
```

This prints:

```
generic      :2
```

Additionally, it doesn't allow formatting the error message and introduces potential encoding issues, as the encoding of the category name is unspecified.

A specifier for an error code's value is intentionally not provided because it is of limited use without the associated category information. Moreover, the value can be easily accessed and formatted using other means, for example:

```
std::print("{}\n", ec.value());
```

This functionality is not currently provided by `{fmt}`, and over several years of usage, there have been no requests to add it. However, if sufficient demand emerges, it could be considered for future inclusion.

§ 8. Proposal

This paper proposes adding a `formatter` specialization for `std::error_code` to address the problems discussed in the previous section.

The default format will produce the same output as the `ostream` inserter:

```
std::print("{}\n", ec);
```

Output:

```
generic:2
```

It will correctly handle width and alignment:

```
std::print("{:>12}\n", ec);
```

Output:

```
[  generic:2]
```

Additionally, it will allow formatting the error message:

```
std::print("{:s}\n", ec);
```

Output:

```
No such file or directory
```

(The actual message depends on the platform.)

The main challenge lies in the standard's lack of specification for the encodings of strings returned by `error_category::name` and `error_code::message` / `error_category::message` ([syserr.errcat.virtuals](#)):

```
virtual const char* name() const noexcept = 0;
```

Returns: A string naming the error category.

```
virtual string message(int ev) const = 0;
```

Returns: A string that describes the error condition denoted by `ev`.

In practice, implementations typically define category names as string literals, meaning they are in the ordinary literal encoding.

However, there is significant divergence in message encodings. `libc++` and `libstdc++` use `strerror[_r]` for the generic category which is in the C (not "C") locale encoding but disagree on the encoding for the system category: `libstdc++` uses the Active Code Page (ACP) while `libc++` again uses `strerror` / C locale on Windows. Microsoft STL uses a table of string literals in the ordinary literal encoding for the generic category and ACP for the system category.

The following table summarizes the differences:

	libstdc++	libc++	Microsoft STL
POSIX	<code>strerror</code>	<code>strerror</code>	N/A
Windows	<code>strerror</code> / ACP	<code>strerror</code>	ordinary literals / ACP

Obviously none of this is usable in a portable way through the generic `error_category` API because encodings can be and often are different.

To address this, the proposal suggests using the C locale encoding (execution character set), which is already employed in most cases and aligns with underlying system APIs. Microsoft STL's implementation has a number of bugs in `std::system_category::message` ([MSSTL-3254], [MSSTL-4711]) and will likely need to change anyway. This also resolves [LWG4156].

An alternative approach could involve communicating the encoding from `error_category`. However, this introduces ABI challenges and complicates usage compared to adopting a single encoding.

§ 9. Previous work

A formatter for `std::error_code` was proposed as part of [P2930] which has more formatting options for the numeric code but doesn't try to address encoding issues or provide a debug format.

§ 10. Wording

Add to "Header <system_error> synopsis" [system.error.syn]:

```
// [system.error.fmt], formatter
template<class charT> struct formatter<error_code, charT>;
```

Add a new section "Formatting" [system.error.fmt] under "Class `error_code`" [syserr.errcode]:

```
template<class charT> struct formatter<error_code, charT> {
    constexpr void set_debug_format();

    constexpr typename basic_format_parse_context<charT>::iterator
        parse(basic_format_parse_context<charT>& ctx);

    template<class FormatContext>
        typename FormatContext::iterator
            format(const error_code& ec, FormatContext& ctx) const;
};
```

```
constexpr void set_debug_format();
```

Effects: Modifies the state of the `formatter` to be as if the *error-code-format-spec* parsed by the last call to `parse` contained the `?` option.

```
constexpr typename basic_format_parse_context<charT>::iterator
    parse(basic_format_parse_context<charT>& ctx);
```

Effects: Parses the format specifier as a *error-code-format-spec* and stores the parsed specifiers in `*this`.

error-code-format-spec:

*fill-and-align*_{opt} *width*_{opt} `?`_{opt} `S`_{opt}

where the productions *fill-and-align* and *width* are described in [format.string].

Returns: An iterator past the end of the *error-code-format-spec*.

```
template<class FormatContext>
    typename FormatContext::iterator
        format(const error_code& ec, FormatContext& ctx) const;
```

Effects: If the `S` option is used, then:

- If the ordinary literal encoding is UTF-8, then let `msg` be `ec.message()` transcoded to UTF-8 with maximal subparts of ill-formed subsequences substituted with U+FFFD REPLACEMENT

CHARACTER per the Unicode Standard, Chapter 3.9 U+FFFD Substitution in Conversion.

- Otherwise, let `msg` be `ec.message()` transcoded to an implementation-defined encoding.

Otherwise, let `msg` be `format("{}:{}", ec.category().name(), ec.value())`.

Writes `msg` into `ctx.out()`, adjusted according to the *error-code-format-spec*. If the `?` option is used then `msg` is formatted as an escaped string (`[format.string.escaped]`).

Returns: An iterator past the end of the output range.

Modify `[syserr.errcat.virtuals]`:

```
virtual const char* name() const noexcept = 0;
```

Returns: A string in the ordinary literal encoding naming the error category.

...

```
virtual string message(int ev) const = 0;
```

Returns: A string of multibyte characters in the execution character set that describes the error condition denoted by `ev`.

§ 11. Implementation

The proposed `formatter` for `std::error_code` has been implemented in the open-source `{fmt}` library (`[FMT]`).

§ References

§ Informative References

[FMT]

Victor Zverovich; et al. *The {fmt} library*. URL: <https://github.com/fmtlib/fmt>

[LWG4156]

Victor Zverovich. *`error\_category` messages have unspecified encoding*. URL: <https://cplusplus.github.io/LWG/issue4156>

[MSSTL-3254]

Visual Studio 2022 std::system_category returns "unknown error" if system locale is not en-US.

URL: <https://github.com/microsoft/STL/issues/3254>

[MSSTL-4711]

Sung Po-Han. *Should `std::error\_code::message` respect the locale set by the user?.* URL:

<https://github.com/microsoft/STL/issues/4711>

[P2930]

Mark de Wever. *Formatter specializations for the standard library.* URL: <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/p2930r0.html>